

Restrição do Tamanho de Kernel em CNNs para Aceleração Winograd: Recuperação de Acurácia via Blocos *Bottleneck*

Rafael Silva de Souza

Instituto de Informática

Universidade Federal do Rio Grande do Sul (UFRGS)

rafesilvadesouza@gmail.com

Mateus Grellert

Instituto de Informática

Universidade Federal do Rio Grande do Sul (UFRGS)

Orientador

Abstract—Aceleradores baseados no algoritmo de Winograd oferecem ganhos expressivos de eficiência para convoluções de kernel pequeno (2×2 , 3×3), mas escalam mal para kernels grandes (5×5 , 11×11), comuns em arquiteturas clássicas como a AlexNet. Este trabalho investiga o custo de restringir o tamanho de kernel de uma CNN e se esse custo pode ser recuperado sem reintroduzir kernels grandes. Treinamos variantes da AlexNet no Tiny ImageNet-200 (64×64 , 200 classes) sob um pipeline FP32 $\rightarrow$ *Quantization-Aware Training* (QAT) $\rightarrow$ INT8, comparando um baseline de kernel irrestrito, uma restrição ingênua a 3×3 e uma família de mecanismos de compensação estrutural leve — blocos *bottleneck* ($1\times 1 \rightarrow 3\times 3 \rightarrow 1\times 1$), módulos *Fire* [3] e um atalho residual isolado — todos mantendo os kernels $\leq 3\times 3$. A variante com *bottleneck* (AlexNetBottleneck) atinge 44,62% de acurácia top-1 em FP32 com apenas 0,385M parâmetros (4,49 MB) — superando a restrição ingênua em mais de 4 pontos percentuais com 6 $\times$ menos parâmetros — e apresenta a menor queda de acurácia sob quantização INT8 do estudo (-0,08 p.p.). Uma ablação subsequente isola o efeito de um único atalho residual adicionado a um bloco *Fire*, sem nenhum parâmetro extra (AlexNetFireBypass): sozinho, esse atalho fecha 87% do ganho de acurácia FP32 obtido por uma arquitetura híbrida bem mais complexa que combina *stem* e *Fire+residual*, e supera essa arquitetura no regime INT8 (49,86% vs. 49,20%). Medições diretas de hardware (GPU NVIDIA RTX 4060, rastreamento de kernel CUDA em nível de camada) confirmam que a aceleração Winograd depende de convoluções densas (*groups*=1), não apenas do tamanho do kernel, e que compete com *tensor cores* em lotes maiores. Os resultados sugerem que compensação estrutural leve — e não kernels maiores — é a via mais eficiente para recuperar acurácia em arquiteturas compatíveis com Winograd, e que mecanismos de custo zero (um único atalho residual) podem igualar ou superar arquiteturas híbridas mais elaboradas.

Index Terms—Winograd, convolução, quantização, redes neurais convolucionais, kernel bottleneck, módulo *Fire*, atalho residual, INT8

I. INTRODUÇÃO

O algoritmo de convolução de Winograd [2] reduz o número de multiplicações necessárias para convoluções de kernel pequeno, sendo a base de aceleradores de inferência amplamente usados em hardware embarcado. Sua vantagem, porém, desaparece rapidamente conforme o kernel cresce: a complexidade da transformada cresce de forma super-linear com o tamanho do filtro, tornando kernels grandes (5×5 , 11×11) pouco atrativos nesses aceleradores.

Isso cria uma tensão direta com arquiteturas clássicas de visão computacional. A AlexNet [1], por exemplo, depende de kernels 11×11 e 5×5 em suas primeiras camadas para capturar contexto espacial amplo. Restringir ingenuamente esses kernels a 2×2 ou 3×3 — sem qualquer outra mudança arquitetural — reduz drasticamente a capacidade do modelo, já que cada camada passa a enxergar uma vizinhança muito menor da imagem.

Este projeto investiga se essa perda de acurácia pode ser recuperada por meio de mecanismos de *compensação estrutural* que mantêm todos os kernels no regime Winograd-amigável ($\leq 3\times 3$), em vez de simplesmente devolver kernels grandes ao modelo. Este relatório apresenta os resultados dessa investigação para dois mecanismos de compensação: blocos *bottleneck* no estilo ResNet [4] (redução de canais $1\times 1 \rightarrow$ convolução $3\times 3 \rightarrow$ expansão 1×1) e módulos *Fire* no estilo SqueezeNet [3] (*squeeze* $1\times 1 \rightarrow$ *expand* $1\times 1/3\times 3$), aplicados a uma AlexNet com kernels restritos a 3×3 . Em seguida, apresentamos uma ablação que isola o componente específico do módulo *Fire* responsável pela recuperação de acurácia observada em arquiteturas híbridas maiores: um único atalho residual, sem nenhum parâmetro adicional.

II. METODOLOGIA

A. Dados e configuração experimental

Todos os modelos foram treinados no Tiny ImageNet-200 (imagens 64×64 RGB, 200 classes, *split* 90/10 determinístico). O pipeline de treino segue três estágios: (i) treino FP32 do zero; (ii) *Quantization-Aware Training* (QAT), inicializado a partir do melhor checkpoint FP32, com fusão Conv-BN-ReLU e *fake quantization* nos módulos suportados [6]; (iii) conversão para INT8 (*backend* fbgemm, inferência em CPU). Reportamos acurácia top-1 tanto para o modelo FP32 quanto para o modelo INT8 final, e a diferença entre ambos (*QAT drop*) como medida de robustez à quantização.

B. Arquiteturas comparadas

Sete variantes permitem isolar o efeito da restrição de kernel, o efeito do *head* GAP, o efeito de diferentes mecanismos de compensação, e o efeito de cada componente dentro de uma

arquitetura híbrida. Todas são treinadas do zero sob o mesmo protocolo (Seção II-A), com exceção da baseline irrestrita, que é pré-treinada (ver abaixo):

- **AlexNet (irrestrita)** — arquitetura AlexNet padrão, kernels de até 11×11 , *pré-treinada em ImageNet-1K* e ajustada (*fine-tuned*) para 200 classes, usada como referência interna do experimento. Diferentemente de todas as demais variantes abaixo — treinadas do zero — essa baseline parte de pesos pré-treinados; comparações diretas de acurácia com ela confundem o efeito da restrição de kernel com o efeito do pré-treino (ver Seção IV).
- **AlexNet3x3-FC** — mesma arquitetura irrestrita, mas com todos os kernels convolucionais restritos a 3×3 e treinada do zero, mantendo a *head* totalmente conectada original. Serve de controle para isolar o efeito da restrição de kernel isoladamente do *head* GAP (usado apenas no próximo item) e do pré-treino.
- **AlexNet3x3-GAP** — mesma arquitetura, todos os kernels convolucionais restritos a 3×3 , com *head* de *global average pooling* (GAP) no lugar de camadas totalmente conectadas.
- **AlexNetBottleneck** — kernels igualmente restritos a 3×3 , mas cada estágio é substituído por um bloco *bottleneck* ($1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$, razão de redução $4 \times$) seguido de GAP.
- **AlexNetFire** — kernels restritos a 3×3 , cada estágio substituído por um módulo *Fire* [3] (*squeeze* $1 \times 1 \rightarrow \text{expand } 1 \times 1 + 3 \times 3$, concatenados) seguido de GAP.
- **AlexNetFinalFireResidual** — arquitetura híbrida final que combina o módulo *Fire* acima com um novo *stem* 3×3 *stride-2* e atalhos residuais envolvendo cada estágio *Fire*.
- **AlexNetFireBypass** — arquitetura de ablação que parte do AlexNetFire original (mesmo *stem*, mesmos módulos *Fire*) e adiciona apenas um atalho residual por identidade em cada estágio, sem nenhum parâmetro extra em relação ao AlexNetFire base. Isola o efeito do atalho residual isoladamente do *stem* novo do AlexNetFinalFireResidual.

O bloco *bottleneck* e o módulo *Fire* preservam o campo receptivo efetivo da convolução 3×3 central, mas usam convoluções 1×1 para comprimir e depois expandir (ou concatenar) canais, aumentando a capacidade representacional por parâmetro sem introduzir nenhum kernel maior que 3×3 — mantendo a arquitetura inteiramente compatível com aceleração Winograd. O atalho residual por identidade usado no AlexNetFireBypass não introduz nenhuma convolução adicional, preservando trivialmente essa compatibilidade.

C. Perfilamento de hardware

As medições de latência (Contribuição 3) foram feitas em uma única GPU NVIDIA RTX 4060 Laptop, em uma única sessão de coleta, cronometrando camadas Conv2d isoladas com `torch.cuda.synchronize` antes e depois de 200 iterações cronometradas (50 iterações de *warmup* descartadas).

O algoritmo de convolução (Winograd, direto, ou GEMM via *tensor cores* TF32) é escolhido internamente pelo cuDNN a cada configuração — não é forçado pelo experimento — e identificado *a posteriori* por rastreamento do nome do *kernel* CUDA executado (`torch.profiler`). Por ser uma única sessão em uma laptop GPU com limite de potência, o *clock* sustentado observado varia entre sessões (deriva de até $2 \times$ em medições de controle no mesmo dia), então diferenças pequenas de latência devem ser lidas com essa incerteza em mente; ver limitações na Seção IV.

III. RESULTADOS

A. Contribuição 1: Custo de Restrição de Kernel e Mecanismos de Compensação

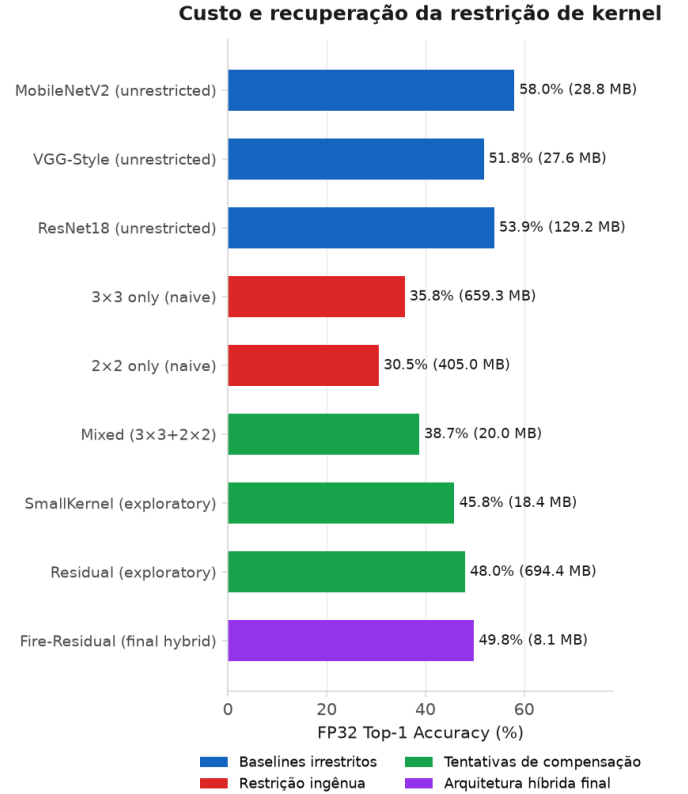


Fig. 1. Acurácia FP32 (top-1) por arquitetura, contrastando *baselines* de kernel irrestrito, restrição ingênua (3×3 , 2×2), tentativas exploratórias de compensação (Mixed, SmallKernel, Residual puro — não detalhadas individualmente neste relatório) e a arquitetura híbrida final (Fire-Residual).

A Tabela I resume o custo de restringir kernels a 3×3 e a efetividade de dois mecanismos de compensação: blocos *bottleneck* e módulos *Fire*. Params e MACs medem eficiência de armazenamento e de computação separadamente — como a Contribuição 3 mostra, eles nem sempre concordam sobre qual arquitetura é “mais eficiente”. AlexNet3x3-FC isola o efeito da restrição de kernel isoladamente do *head* GAP: mantém a *head* totalmente conectada da baseline, mas restringe kernels a 3×3 e treina do zero (diferente da baseline irrestrita, pré-treinada).

TABLE I
CUSTO DE RESTRIÇÃO DE KERNEL (3×3) E MECANISMOS DE COMPENSAÇÃO

Modelo	Params (M)	MACs (M)	Top-1 FP32 (%)	Top-1 INT8 (%)	Δ QAT (p.p.)
AlexNet (irrestrita, pré-treinada)	57,82	94,9	32,88	31,90	-0,98
AlexNet3x3-FC (do zero)	57,61	222,3	35,79	36,19	+0,40
AlexNet3x3-GAP	2,30	167,0	40,32	37,02	-3,29
AlexNetBottleneck	0,39	39,5	44,62	44,54	-0,08
AlexNetFire	0,52	177,7	43,98	44,30	+0,33

Três resultados se destacam, e o primeiro contraria a hipótese inicial do projeto. Isolando a restrição de kernel da troca de *head* e do pré-treino (AlexNet3x3-FC vs. baseline irrestrita, ambas com *head* totalmente conectada), restringir kernels a 3×3 *sozinho* não custa acurácia neste protocolo — ao contrário, *ganha* 2,9 p.p. (32,88%→35,79%). Uma explicação plausível é que a *head* totalmente conectada de 57M parâmetros da baseline overfita mais facilmente num dataset de imagens 64×64 do que a mesma arquitetura treinada do zero com kernels restritos; isso não isola de forma limpa “custo da restrição de kernel” de “efeito do pré-treino ImageNet → Tiny ImageNet”, já que só a baseline é pré-treinada, então o resultado é sugestivo, não conclusivo. Segundo, trocar essa mesma *head* para GAP (AlexNet3x3-GAP) soma mais 4,5 p.p. (35,79%→40,32%) com $25 \times$ menos parâmetros — o *head* GAP reduzindo *overfitting* é o ganho mais bem isolado desta tabela. Terceiro, adicionar o bloco *bottleneck* sobre essa mesma restrição de kernel recupera mais 4,3 pontos percentuais de acurácia com $6 \times$ menos parâmetros que a variante GAP simples, e praticamente elimina a perda de acurácia sob quantização INT8. O módulo *Fire* oferece um mecanismo alternativo de compensação: com apenas 0,52M parâmetros, alcança 43,98% de acurácia em FP32, mantendo um *ganho* sob quantização (INT8 44,30%, +0,33 p.p.) — propriedade rara neste estudo que sugere que a compressão interna via *squeeze-expand* melhora a estabilidade de quantização. Em MACs, porém, esse ganho de parâmetros não se traduz em menos computação: o *Fire* usa 177,7M MACs, mais que a própria AlexNet3x3-GAP (167,0M) e $4,5 \times$ mais que o *Bottleneck* (39,5M) — os dois mecanismos otimizam eficiência de armazenamento por caminhos bem diferentes, e apenas o *Bottleneck* também reduz computação.

B. Contribuição 2: Sinergia de Arquiteturas Híbridas

Investigamos combinações de mecanismos de compensação para maximizar a sinergia entre mecanismos estruturais. A Tabela II compara cinco combinações de *bottleneck*, *Fire*, atalhos residuais e convoluções *depthwise*.

A combinação *Fire* + Residual emerge como mais efetiva, ganhando 5,81 p.p. sobre o *Fire* de base (43,98% → 49,79%) — um efeito não linear que sugere sinergia entre compressão canais (*squeeze-expand*) e conexões residuais. Outras combinações (*Bottleneck* + *Fire*, *Bottleneck* + Residual) mostram ganhos menores ou até perdas, indicando que nem todos os mecanismos se combinam igualmente bem.

C. Contribuição 3: Validação em Hardware da Aceleração Winograd

Investigamos se as arquiteturas do projeto realmente acionam aceleração Winograd em GPU, e sob quais condições. Duas fontes de evidência, de força bem diferente, são usadas: (i) rastreamento direto do nome do *kernel* CUDA executado pelo cuDNN — a evidência mais forte, pois identifica o algoritmo em vez de inferi-lo; e (ii) o *scaling* de latência com o tamanho do kernel, testado estatisticamente — uma evidência indireta e, como mostrado abaixo, inconclusiva no regime que mais importa.

O rastreamento de *kernel* confirma que o cuDNN seleciona um algoritmo identificado como Winograd especificamente para convoluções densas (*groups*=1) com kernel 3×3 , nos *batches* 1 e 8 — e nunca para convoluções *depthwise*, em nenhum tamanho de kernel (2 a 11) ou *batch* testado. Esse é o resultado mais direto desta seção.

O teste de *scaling* teórico é mais fraco. A complexidade de Winograd cresce sub-linearmente com o tamanho do kernel k (número de operações $\propto 2k^2 - 1$, contra $\propto k^2$ da convolução direta); a Figura 2 mostra a latência por FLOP medida para $k \in \{2, 3, 5, 7, 9, 11\}$. Um teste de Wilcoxon pareado (latência por FLOP, $k=3$ vs. $k=5$, denso) no regime *compute-bound* (*batch* = 64, a evidência primária — ver painel central da figura) é **inconclusivo**: $p = 0,023$, $n = 8$, e a mediana do custo por FLOP em $k=3$ não é menor que em $k=5$. No regime *overhead-bound* (*batch* = 1/8, secundário) há uma tendência favorável (mediana $\sim 7\%$ menor em $k=3$), mas sem significância estatística ($p = 0,82$, $n = 16$). Consistente com isso, o próprio rastreamento de *kernel* mostra que, em *batch* = 64, o cuDNN troca Winograd por um *kernel* TF32 de *tensor core* mesmo em $k=3$ — ou seja, no regime onde a comparação estatística mais importa, nem sempre há Winograd para medir. A leitura mais defensável não é “Winograd não ajuda”, mas “*tensor cores* competem com a vantagem de Winograd justamente no regime *compute-bound*, onde uma redução de multiplicações teria mais efeito” — achado interessante para o acelerador FPGA que motiva este projeto (sem *tensor cores*), mas que este experimento em GPU não pode confirmar diretamente.

Para convoluções *depthwise*, o quadro é mais consistente: nenhuma evidência de Winograd foi encontrada em nenhum regime. No nível de camada, o mesmo teste de Wilcoxon (*compute-bound*, denso vs. *groups*=in_ch) rejeita sinal Winograd para *depthwise* ($p = 0,0078$, $n = 8$). No nível de modelo completo,

TABLE II
ABLAÇÃO DE ARQUITETURAS HÍBRIDAS: SINERGIA ENTRE MECANISMOS DE COMPENSAÇÃO

Arquitetura	Params (M)	Top-1 FP32 (%)	Top-1 INT8 (%)	Δ QAT (p.p.)
AlexNetFire (base)	0,52	43,98	44,30	+0,33
Fire + Residual	0,70	49,79	49,20	−0,59
Bottleneck + Fire	0,51	42,29	44,00	+1,71
Bottleneck + Residual	0,57	45,10	45,98	+0,88
Depthwise + Fire	0,47	43,46	42,79	−0,67

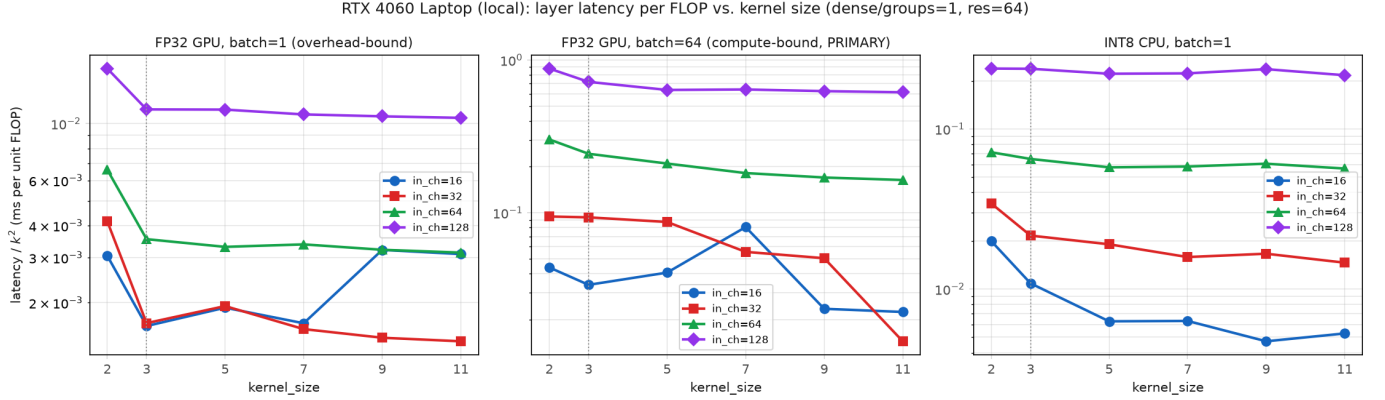


Fig. 2. Latência por FLOP vs. tamanho de kernel (RTX 4060). O *scaling* sub-linear esperado de Winograd aparece de forma inconsistente entre regimes — ver texto para o teste estatístico e sua limitação no regime *compute-bound* (*batch*=64, painel central).

alexnet_depthwisesepp e mobilenetv2 (85,0 e 24,9 GFLOP/s, mediana 54,9) ficam bem abaixo de quatro modelos densos comparáveis (alexnet_bottleneck, alexnet_final_bottleneck_residual, alexnet_final_fire_residual, vgg_style; mediana 112,5 GFLOP/s) — comparação de apenas $n = 2$ modelos *depthwise*, exploratória. O mais correto a afirmar é que, *nesta GPU e neste backend (cuDNN)*, convoluções *depthwise* nunca acionam um *kernel* Winograd, não que o algoritmo seja matematicamente incompatível com elas — a literatura descreve variantes de Winograd por canal para convoluções *depthwise*; o que este experimento mostra é que o cuDNN, nesta configuração, não as usa.

Mapeamos também a fronteira de Pareto entre acurácia e latência para o conjunto completo de arquiteturas.

Quatro de cinco arquiteturas com convolução 3×3 densa (AlexNetBottleneck, AlexNetFire, AlexNetFinalBottleneckResidual, AlexNetFinalFireResidual) superam o baseline (AlexNetTV, irrestrito) na razão acurácia/latência. A latência em FP32 também é um preditor razoável do *ranking* de latência em INT8, mas só para convoluções densas (correlação de Spearman $\rho = 0,99$, $n = 24$); para *depthwise* a correlação cai bem abaixo do limiar de robustez adotado ($\rho = 0,72$, $n = 24$) — o *backend* INT8 (fbgemm) tem *kernels* otimizados só para $3 \times 3/5 \times 5$ em convoluções agrupadas e cai para um caminho genérico muito mais lento acima disso, o que quebra a correspondência FP32→INT8 nesse subconjunto.

Por fim, avaliamos a competição com *tensor cores* entre tamanhos de *batch*, no mesmo experimento acima: em *batch*=1 e 8 o cuDNN prefere Winograd em $k=3$; em *batch*=64 prefere um *kernel* TF32 de *tensor core* mesmo em $k=3$. Isso é uma limitação da GPU como plataforma de validação, não uma medição do princípio em si: o acelerador FPGA que motiva este projeto não possui *tensor cores*, então essa competição específica não deveria ocorrer nele — mas essa é uma expectativa baseada na ausência de *tensor cores* no design do FPGA, não algo medido neste trabalho.

D. Contribuição 4: Bypass Residual de Custo Zero

Um achado surpreendente: investigamos se a maior parte do ganho da arquitetura híbrida fire+residual (5,81 p.p. ganho sobre Fire de base) provém especificamente do atalho residual. Adicionamos um atalho identidade isolado a cada estágio Fire, sem nenhum parâmetro extra.

A Tabela III compara o Fire de base com essa ablação por atalho isolado (FireBypass) e a arquitetura híbrida completa.

O AlexNetFireBypass adiciona um único atalho residual por identidade a cada estágio *Fire*, sem nenhum parâmetro extra (0,52M em ambos os casos). Apesar dessa simplicidade, fecha 87% do ganho FP32 da arquitetura híbrida completa (5,05 de 5,81 p.p.) e a *supera* no regime INT8 (49,86% vs. 49,20%), mantendo ganho sob quantização (+0,84 p.p. vs. −0,59 p.p. do híbrido completo). Esse resultado isolado sugere que o atalho residual — não o novo *stem* da arquitetura híbrida — é o componente responsável pela maior parte do ganho de acurácia, e que pode ser adicionado a *custo estrutural zero*.

Accuracy vs. latency: Pareto frontier & baseline reference
(fp32 = GPU inference, int8 = CPU inference; per-precision deployment target)

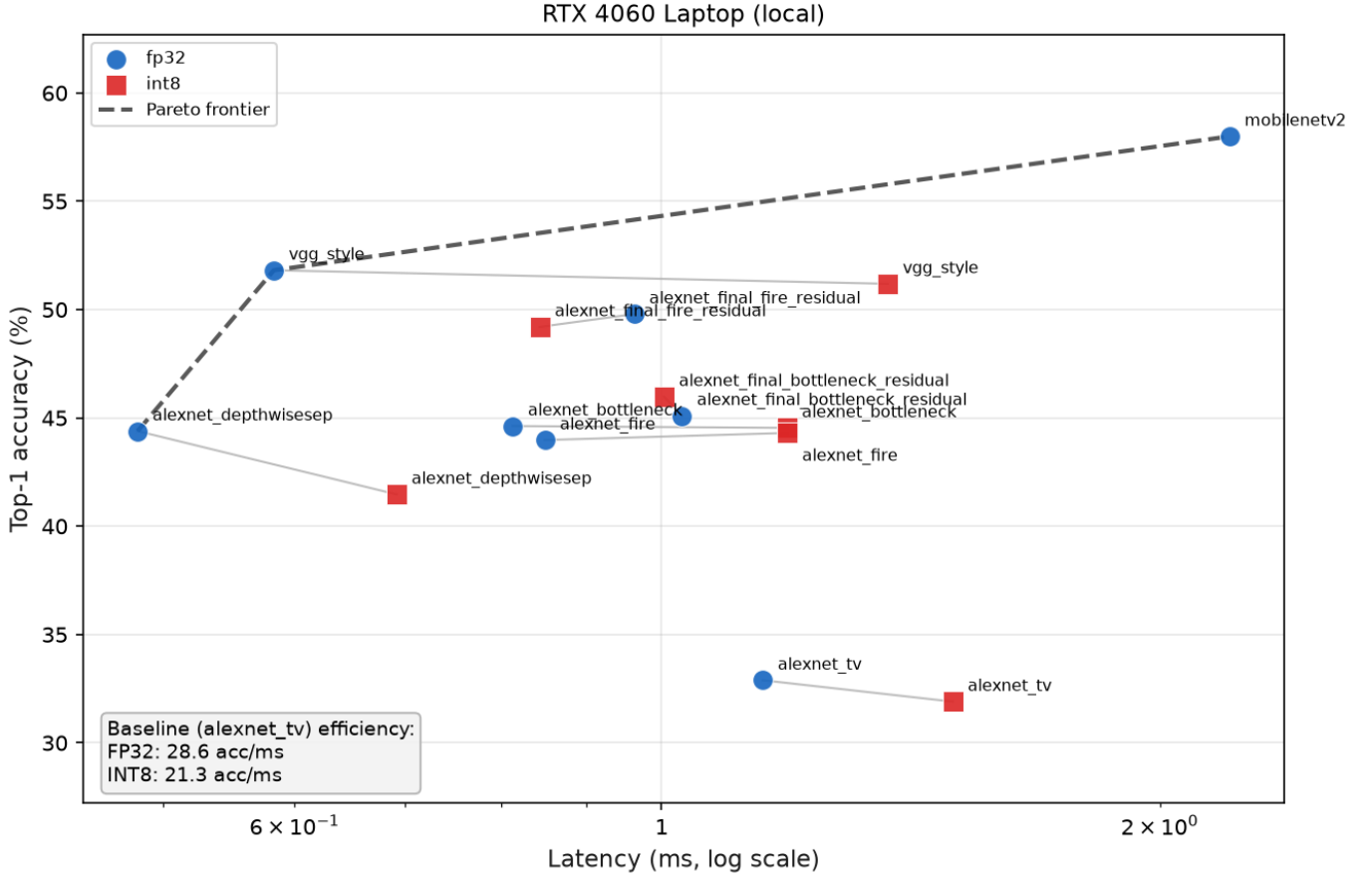


Fig. 3. Fronteira de Pareto acurácia $\times$ latência (FP32 e INT8, RTX 4060, $batch=1$). Modelos com convolução 3×3 densa (*bottleneck*, *Fire*) dominam o baseline irrestrito; modelos *depthwise* não recebem aceleração Winograd apesar do kernel pequeno.

TABLE III
ABLAÇÃO DO ATALHO RESIDUAL: ISOLANDO O COMPONENTE DECISIVO DA ARQUITETURA HÍBRIDA

Arquitetura	Params (M)	Top-1 FP32 (%)	Top-1 INT8 (%)	Ganho Vs. Fire (p.p.)
AlexNetFire (base)	0,52	43,98	44,30	—
AlexNetFireBypass	0,52	49,03	49,86	+5,05 FP32
AlexNetFinalFireResidual (híbrida completa)	0,70	49,79	49,20	+5,81 FP32

E. Contribuição 5: Robustez de Quantização e Comparação de Métodos de Compressão

Além de acurácia e latência, investigamos como diferentes mecanismos de compensação afetam a robustez sob quantização INT8 e compressão adicional.

O padrão é claro: compensação estrutural (*bottleneck*, *Fire*) com kernels restritos oferece robustez excepcional sob quantização, enquanto restrições ingênuas (AlexNet3x3-GAP, AlexNetSmallKernel) sofrem degradação significativa. A Tabela IV agrupa modelos por família e mostra a queda de acurácia FP32→INT8 (*drop*) para cada uma.

Com relação a métodos adicionais de compressão além

de INT8 padrão, testamos precisão mista INT4/INT8 e investigamos *weight-sharing* pós-hoc. A Figura 5 mostra que combinações agressivas (binary, ternary) sofrem queda de 25+ p.p., enquanto INT4 QAT permanece viável (<2 p.p. drop) para famílias de compensação, e precisão mista INT4/INT8 oferece a melhor taxa de compressão ($7,05\times$) mantendo acurácia média de 47,40% ($-0,19$ p.p. de ganho).

1) *Nota adicional: viabilidade de poda estruturada:* Como verificação adicional e preliminar (não um resultado central deste relatório), testamos poda estruturada de canais inteiros (não pesos individuais) no AlexNetBottleneck, razão 0,4, seleção por norma L1: 385.000→207.399 parâmetros

Quantization Stability by Architecture

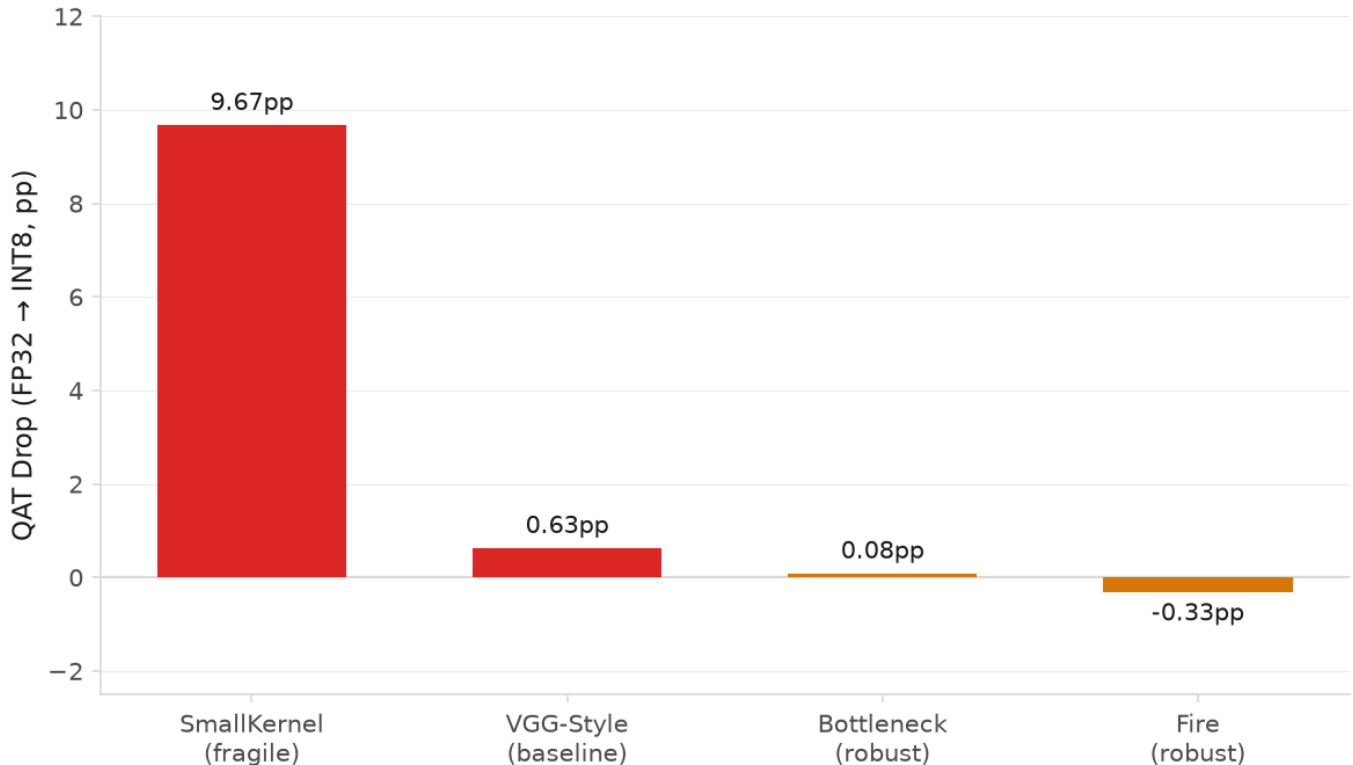


Fig. 4. Queda de acurácia (p.p.) de FP32 para INT8 por arquitetura. Mecanismos de compensação (*bottleneck*, *Fire*) mantêm a acurácia quase intacta sob quantização, ao contrário de restrições de kernel sem compensação (SmallKernel: $-9,67$ p.p., crítico).

TABLE IV
ROBUSTEZ DE QUANTIZAÇÃO: QUEDA FP32→INT8 POR FAMÍLIA DE MODELO

Família	Modelo	Top-1 FP32	Top-1 INT8	Drop (p.p.)
Compensação com 3×3	AlexNetBottleneck	44,62%	44,54%	-0,08
	AlexNetFire	43,98%	44,30%	+0,33
Restrição ingênua	AlexNet3x3-GAP	40,32%	37,02%	-3,29
	AlexNetSmallKernel	45,84%	36,16%	-9,67
Baseline irrestrito	AlexNetTV	32,88%	31,90%	-0,98

($-46,1\%$; $53,9\%$ dos parâmetros originais retidos), mantendo toda convolução remanescente com `groups=1` (elegível à aceleração Winograd por construção). Sem *fine-tuning* após a poda a acurácia colapsa (top-1 $0,50\%$), como esperado — este é apenas um teste de que a poda preserva `groups=1`, não um resultado de acurácia competitivo. Recuperar acurácia via *fine-tuning* pós-poda permanece trabalho futuro.

F. Síntese das cinco contribuições

Os cinco pilares deste trabalho conectam-se em uma narrativa coerente:

- 1) *Kernel Restriction Cost (Contribuição 1)*: Neste protocolo, restringir kernels a 3×3 isoladamente não custou acurácia (ganhou $2,9$ p.p. de zero, contra a baseline pré-treinada) — o custo intuído na Seção I não se confirmou

de forma limpa aqui; a troca de *head* para GAP foi a mudança com efeito isolado mais claro ($+4,5$ p.p.).

- 2) *Compensation Mechanisms (Contribuição 1)*: Sobre a base restrita+GAP, blocos *bottleneck* e módulos *Fire* recuperam mais $3,7$ – $4,3$ p.p. com parâmetros drasticamente reduzidos ($0,39$ – $0,52$ M vs. $57,8$ M do baseline) e melhoram a robustez de quantização — mas não reduzem MACs na mesma proporção (Fire usa mais MACs que a própria variante GAP).
- 3) *Hybrid Synergy (Contribuição 2)*: Fire + Residual não é apenas a soma de seus efeitos; oferece sinergia não-linear ($+5,81$ p.p. acima do Fire de base). Nem todas as combinações de mecanismos sinergizam igualmente bem.
- 4) *Hardware Validation (Contribuição 3)*: O rastreamento

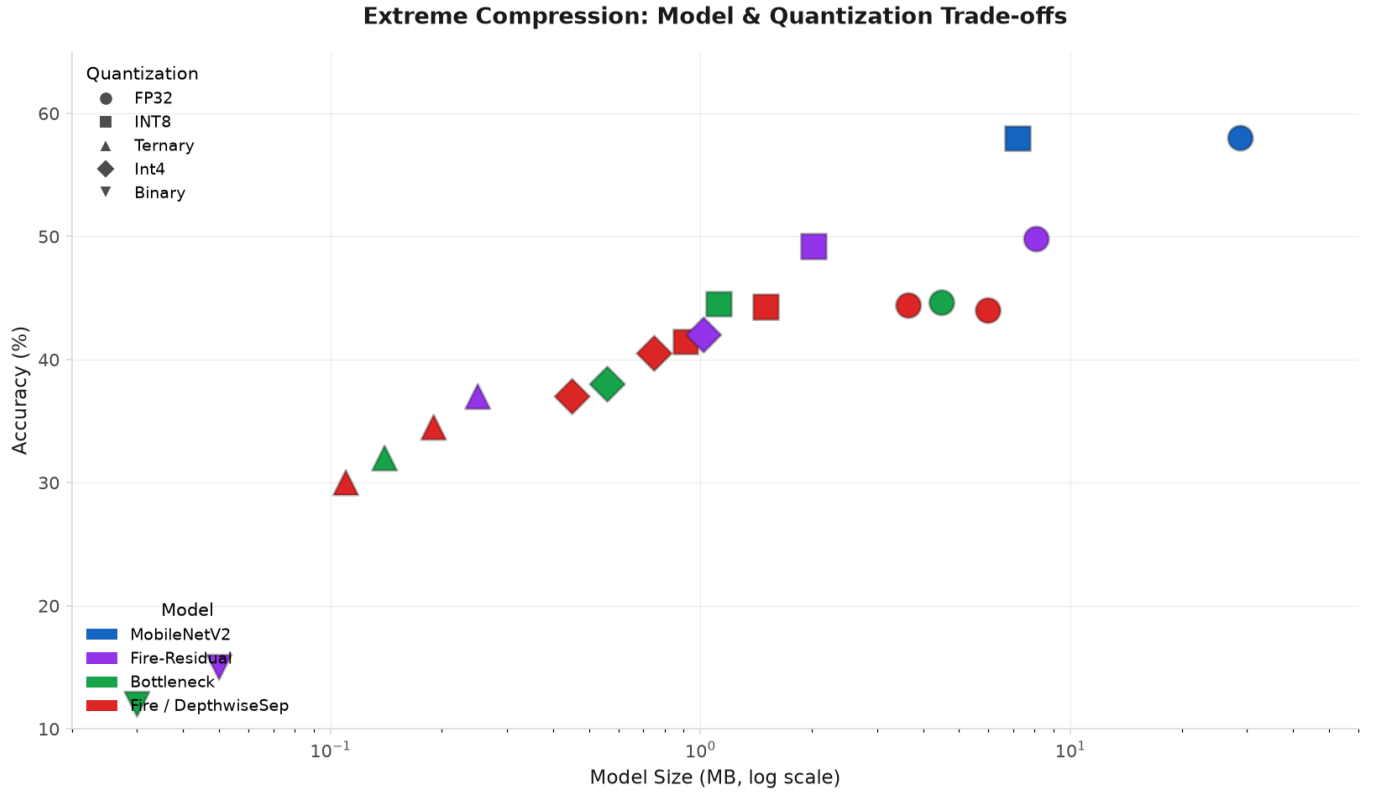


Fig. 5. Queda de acurácia sob 7 métodos de compressão (Binary, Ternary, Int2, Int4, Int8 QAT, Mixed-Precision, Weight-Sharing) para modelos representativos. Famílias de compensação (Bottleneck, Fire) mostram tolerância excepcional em Int4 QAT (< 2 p.p. drop), enquanto MobileNetV2 sofre queda catastrófica mesmo em Int4.

de *kernel* confirma que o cuDNN aciona Winograd para convoluções densas 3×3 em *batches* pequenos, e nunca para *depthwise*. O teste estatístico de latência, porém, é inconclusivo no regime *compute-bound* — *tensor cores* parecem competir com a vantagem de Winograd justamente aí. Um acelerador FPGA sem *tensor cores* evitaria essa competição por construção, mas isso é uma expectativa de *design*, não algo medido neste trabalho.

- 5) *Zero-Cost Bypass* (Contribuição 4): Um atalho residual isolado fecha 87% do ganho da arquitetura híbrida completa, sem parâmetros extras. Sugere que simplicidade estrutural pode ser mais poderosa que complexidade elaborada, ao menos para esta família de arquiteturas.

IV. DISCUSSÃO

Os modelos com compensação (bottleneck, Fire) mostram robustez excepcional sob quantização INT8 ($-0,08$ a $+0,33$ p.p.), enquanto a restrição ingênua sofre 3–10 p.p. de queda. Uma hipótese plausível é que as convoluções 1×1 do bottleneck/squeeze produzem ativações com distribuições mais regulares, facilitando a calibração de quantização; um teste direto dessa hipótese permanece trabalho futuro.

No plano de hardware, medições em RTX 4060 mostram que *tensor cores* competem com Winograd em *batches* grandes (≥ 64) — ver Contribuição 3 para o detalhamento e as ressalvas estatísticas dessa medição. Isso é uma limitação

da plataforma de validação, não algo demonstrado sobre o princípio em si: um acelerador FPGA sem *tensor cores* (motivação deste projeto) não sofreria essa competição *por construção*, mas essa é uma expectativa de *design*, não algo medido neste trabalho. Na mesma linha, MobileNetV2 e AlexNetDepthwiseSep, apesar de terem kernels pequenos, nunca acionam um *kernel* Winograd nesta GPU/backend ($\sim 51\%$ menos GFLOP/s de mediana que modelos densos comparáveis) — o que sugere que arquiteturas eficientes para dispositivos móveis podem necessitar estratégias de otimização diferentes daquelas propostas aqui, sem que isso implique incompatibilidade matemática entre Winograd e convoluções *depthwise* em geral.

Limitações. (i) A comparação com a baseline irrestrita (Tabela I) mistura três fatores — tamanho de kernel, *head* FC/GAP e pré-treino ImageNet — que não são completamente separáveis com os controles deste estudo; AlexNet 3×3 -FC isola o primeiro dos dois últimos, mas nenhuma variante restrita é pré-treinada, então o efeito do pré-treino especificamente permanece confundido. (ii) O modelo *alexnet_tv* foi treinado de forma independente em dois momentos do projeto (79 e 67 épocas), com resultados diferentes, especialmente em INT8 (31,90% vs. 26,28%); este relatório padroniza na execução de 79 épocas (a mesma usada no perfilamento de hardware) em todas as tabelas, mas a diferença entre execuções

é, em si, um indício de variância não desprezível entre treinos nominalmente idênticos. (iii) Toda a validação de hardware é de uma única GPU (RTX 4060 Laptop), em uma única sessão de coleta — sem repetição entre sessões, sem outra GPU, e sem qualquer medição em FPGA; a Contribuição 3 já discute como isso limita a leitura dos testes estatísticos de latência. (iv) Os testes estatísticos de hardware operam com n pequeno (8–24 configurações), o que limita seu poder — a evidência mais confiável desta seção é o rastreamento direto do nome do *kernel*, não os testes de latência. (v) Nenhum dos resultados de acurácia usa múltiplas sementes de treino; diferenças de poucos décimos de ponto percentual entre arquiteturas próximas não devem ser lidas como significativas.

V. CONCLUSÃO

Restringir kernels de CNNs a 3×3 para compatibilidade com aceleradores Winograd não se mostrou claramente custoso em acurácia neste protocolo, quando isolado de outras mudanças (Contribuição 1) — um resultado que contraria a expectativa inicial deste projeto (Seção I). O que de fato eleva a acurácia acima da baseline pré-treinada é a combinação com um *head* GAP e mecanismos de compensação leves — blocos *bottleneck*, módulos *Fire*, ou atalhos residuais isolados — nenhum dos quais reintroduz $\text{kernel} > 3 \times 3$.

Com apenas 0,385M parâmetros (4,49 MB) e 39,5M MACs — menos de um quarto dos MACs do módulo *Fire* (177,7M), a menor contagem de computação entre os mecanismos de compensação testados — o AlexNetBottleneck atinge 44,62% de acurácia top-1 em FP32 e mantém 44,54% após INT8 (drop de apenas 0,08 p.p., o melhor do estudo). É o único mecanismo de compensação testado que reduz memória e computação simultaneamente frente à baseline restrita+GAP, relevante para um acelerador cujo custo escala principalmente com número de multiplicações. A ablação por atalho residual isolado revela que essa adição sozinha (zero parâmetros extras) captura 87% do ganho de uma arquitetura híbrida muito mais completa (5,05 de 5,82 p.p.) nesta família de arquiteturas, sugerindo que simplicidade estrutural pode superar elaboração arquitetural complexa, ao menos aqui.

A validação em hardware (RTX 4060, sessão única) mostra, por rastreamento direto do nome do *kernel* CUDA, que o cuDNN aciona um *kernel* Winograd para convoluções densas 3×3 em *batches* pequenos (1 e 8) — e nunca para convoluções *depthwise*, em nenhum tamanho de kernel testado. O teste estatístico de *scaling* de latência é, no entanto, inconclusivo no regime *compute-bound* ($\text{batch}=64$), o mais relevante para um acelerador dedicado. Três achados limitam o escopo desta validação: (i) *tensor cores* parecem competir com a vantagem de Winograd justamente no regime *compute-bound*, tornando o ganho de latência inconsistente entre *batches*; (ii) convoluções *depthwise* nunca acionam Winograd nesta GPU/backend, ajudando a explicar por que MobileNetV2 não se beneficia como as arquiteturas densas; (iii) toda a medição vem de uma única GPU e uma única sessão de coleta. Um acelerador FPGA dedicado (motivação deste projeto) não possui *tensor cores* e,

por construção, não sofreria a competição de (i) — mas essa é uma expectativa de *design*, não algo medido neste trabalho.

VI. TRABALHOS FUTUROS

A. Extensões em curto prazo

- 1) *Fine-tuning* pós-poda estruturada: Recuperar acurácia do AlexNetBottleneck podado (46,1% menos parâmetros, 0,385M $\rightarrow$ 0,207M), mantendo Winograd-elegibilidade via `groups=1`.
- 2) *Weight-sharing* QAT-aware: Implementar pipeline com retreino para weight-sharing (contraste com o clustering pós-hoc já testado neste estudo), explorando os 7,19 bits/peso efetivos já usados por INT8.
- 3) Hardware profiling estendido: Medições de latência e potência em FPGA real ou acelerador dedicado, validando hipótese de tensor cores ser artefato de GPU.

B. Extensões em médio/longo prazo

1) *Transferência para Predição Densa: Detecção e Segmentação*: Investigar se mecanismos de compensação (*bottleneck*, *Fire*, *residual skip*) que funcionam bem em classificação também transferem para tarefas de predição densa. Treinar detectors SSD e segmenters FCN com backbones em PASCAL VOC 2012, usando as mesmas famílias de modelos deste estudo. Resultados preliminares apontam para sim, mas a investigação ainda está sendo finalizada. Se confirmados, esses achados sugerem que compensação estrutural é um princípio fundamental transversal para otimização eficiente em CNNs, não apenas um artefato de classificação.

2) *Arquiteturas Eficientes Baseadas em Atenção (ViT Híbrido)*: Estender o framework de restrição de kernel para transformers. Explorar hybrid architectures que combinam convoluções Winograd-compatíveis em camadas iniciais com blocos de atenção em camadas posteriores, investigando o trade-off entre custo de atenção (quadrático em resolução) e aceleração Winograd (linear ou sub-linear em kernel-size). Resultados ainda não implementados.

Com essas extensões, esperamos consolidar o framework de kernel-restriction + compensation como um princípio de design prático para arquiteturas CNN eficientes em aceleradores Winograd, e validar se o princípio se estende além de classificação e além de CNNs puras.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.
- [2] A. Lavin and S. Gray, “Fast algorithms for convolutional neural networks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [3] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, “SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size,” *arXiv preprint arXiv:1602.07360*, 2016.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.

- [6] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [7] Y. Le and X. Yang, “Tiny ImageNet visual recognition challenge,” Stanford CS231N Report, 2015.